



TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN - ĐHQG TP HCM
KHOA TOÁN - TIN HỌC
BỘ MÔN ỨNG DỤNG TIN HỌC

Big O

notation

THỰC HÀNH CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

TUẦN 03 - THÁNG 04 NĂM 2026



Thông tin liên hệ



SAKAI: mcslearn.hcmus.edu.vn

GMAIL: ta.tintth.hcmus@gmail.com

GHI CHÚ:

Tiêu đề mail (bắt buộc):

[CTDL25] [Tiêu đề thư]

VD: [CTDL25] HỎI BÀI

Gửi email kèm giới thiệu họ tên, MSSV
và tên ca học.



Nội Dung

01

*Big O -
Notation*

03

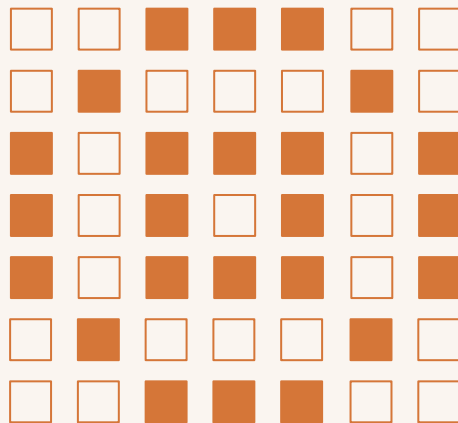
BÀI TẬP TẠI LỚP

02

MỘT SỐ HÀM

04

BÀI TẬP VỀ NHÀ





Big O *notation*

Khái niệm về Big O - notation

Big O - Notation

Big O được sử dụng để đánh giá và so sánh hiệu năng của các thuật toán khác nhau dựa trên tốc độ tăng của thời gian chạy hoặc không gian bộ nhớ khi kích thước đầu vào tăng lên.

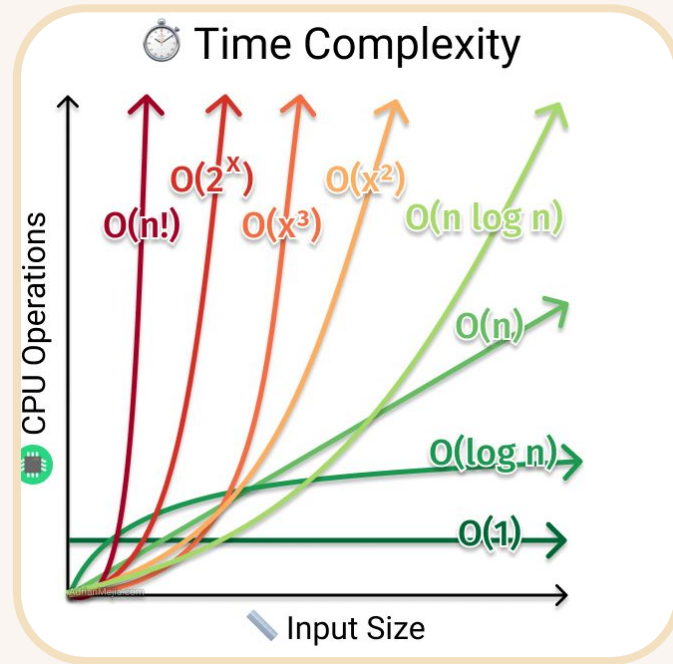
Khi nói về *Big O*, ta thường xem xét cách thuật toán tăng tốc độ thực thi khi kích thước đầu vào tiến tới vô hạn.

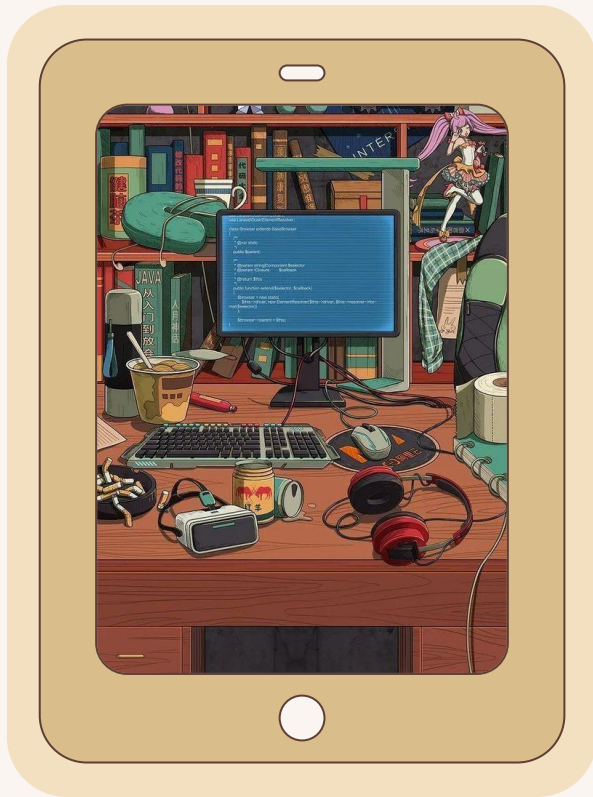
Big O đại diện cho một giới hạn trên (upper bound) cho tốc độ tăng của thuật toán. Nó chỉ quan tâm đến tốc độ tăng tương đối của thuật toán, bỏ qua các hằng số và yếu tố nhỏ khác.



Big O được biểu diễn bằng ký hiệu **$O(f(n))$** , trong đó **$f(n)$** là một hàm mô tả sự tăng của thời gian chạy hoặc không gian bộ nhớ tương đối với kích thước đầu vào n . Các hàm thường được sử dụng trong *Big O* bao gồm:

- **$O(1)$** : Thời gian chạy không đổi, không phụ thuộc vào kích thước đầu vào.
- **$O(\log n)$** : Thời gian chạy tăng theo logarithm cơ số 2 của kích thước đầu vào.
- **$O(n)$** : Thời gian chạy tuyến tính với kích thước đầu vào.
- **$O(n \log n)$** : Thời gian chạy tăng theo tích của kích thước đầu vào và logarithm cơ số 2 của kích thước đầu vào.





02

MỘT SỐ HÀM

Một số hàm gợi ý bổ trợ cho
các bài tập tại lớp.

Hàm sinh mảng số ngẫu nhiên

Công dụng: Sinh một mảng số nguyên có kích thước cho trước (size) trong khoảng min và max.

```
void initializeArray(int arr[], int n, int min, int max) {  
    for (int i = 0; i < n; i++) {  
        arr[i] = min + rand() % (max - min + 1);  
        // Sinh ngẫu nhiên giá trị trong khoảng [min, max]  
    }  
}
```

```
time_t t;  
srand((unsigned)time(&t));  
printf("\n");  
initializeArray(arr, n, min, max); //Sinh mảng
```



Hàm Swap

Công dụng: Đổi chỗ hai giá trị của hai biến.

```
void swap(int *x, int*y){  
    int temp = *x;  
    *x = *y;  
    *y = temp;  
}
```



Hàm Average

Công dụng: Tính trung bình cộng tất cả phần tử trong một mảng số nguyên.

```
float average(int arr[], int n){  
    float sum = 0;  
    for (int i = 0; i < n; i++)  
        sum += arr[i];  
    return sum/n;  
}
```





Linear Search

Công dụng: Tìm kiếm 1 giá trị trong mảng.

```
int linearSearch(int arr[], int n, int value){
    for (int i = 0; i < n; i++){
        if (arr[i] == value)
            return i;
    }
    return -1;
}
```



Binary Search

Công dụng: Tìm kiếm 1 giá trị trong mảng.

```
int binarySearch(int a[], int n, int value, int *compsBinary) {
    int lower_bound = 0;
    int upper_bound = n - 1;
    while (lower_bound <= upper_bound)
    {
        int midpoint = (upper_bound + lower_bound) / 2;
        if (value < a[midpoint]) { upper_bound = midpoint - 1; }
        else if (value > a[midpoint]) { lower_bound = midpoint + 1; }
        else return midpoint;
    }
}
```



Insertion

Công dụng: Chèn 1 giá trị vào trong mảng với chỉ số tương ứng.

```
void insertArray(int arr[], int* n, int value, int index) {  
    if (index < 0 || index > *n)  
        return;  
    (*n)++;  
    for (int i = *n - 1; i > index; i--) {  
        arr[i] = arr[i-1];  
    }  
    arr[index] = value;  
}
```



Deletion

Công dụng: Xoá 1 giá trị vào trong mảng với chỉ số tương ứng.

```
void deleteArray(int a[], int* n, int index) {  
    if (index < 0 || index > (*n)-1) {  
        return;  
    }  
    for (int i = index; i < (*n)-1; i++) {  
        a[i] = a[i + 1];  
    }  
    (*n)--;  
}
```

03

BÀI TẬP TẠI LỚP

Kiểm nghiệm một số thuật
toán đã học ở chương 1 và
chương 2



Linear search

Yêu cầu: Viết chương trình C đếm số bước SO SÁNH trong thuật toán **Linear Search** được cho ở mục 2 (đã học ở CTDLO0 + CTDLO1: Giới thiệu chung)

Lập **bảng tần số** và tính **trung bình cộng** số bước SO SÁNH của thuật toán khi thực hiện trên mảng có kích cỡ cố định **$n = 10$** được sinh ngẫu nhiên, với value là một trong những phần tử của mảng trên được sinh ngẫu nhiên chỉ số, thí nghiệm **k lần** với giá trị của mỗi phần tử nằm trong khoảng $[-1000; 1000]$. Thực hiện yêu cầu với:

- $k = 100$
- $k = 1000$
- $k = 10000$

Chỉnh sửa hàm linear search trên với khai báo:

```
void linearSearch (int arr[], int n, int value, int *comps)
```

Binary search

Yêu cầu: Viết chương trình C đếm số bước thực hiện vòng lặp **LOOPS** trong thuật toán **Binary Search** được cho ở mục 2.

Lập **bảng tần số** và tính **trung bình cộng** số bước **LOOPS** của thuật toán khi thực hiện trên mảng có kích cỡ cố định $n = 10$ được sinh ngẫu nhiên, với value là một trong những phần tử của mảng trên được sinh ngẫu nhiên chỉ số **k lần** với giá trị của mỗi phần tử nằm trong khoảng $[-1000; 1000]$. Thực hiện yêu cầu với:

- $k = 100$
- $k = 1000$
- $k = 10000$

Chỉnh sửa hàm **binarySearch** trên với khai báo:

```
void binarySearch (int arr[], int n, int value, int *loops)
```

Insertion

Yêu cầu: Viết chương trình C đếm số bước **GÁN** trong thuật toán **Insert** được cho ở mục 2

Lập **bảng tần số** và tính **trung bình cộng** số bước **GÁN** của thuật toán khi thực hiện trên mảng có kích cỡ cố định $n = 10$ được sinh ngẫu nhiên, giá trị của mỗi phần tử nằm trong khoảng $[-1000; 1000]$, value cần chèn là **k** và chỉ số cần chèn cũng được sinh ngẫu nhiên từ 0 đến $n-1$. Thực hiện yêu cầu với:

- $k = 100$
- $k = 1000$
- $k = 10000$

Chỉnh sửa hàm **insert** trên với khai báo:

```
void insert(int arr[], int* n, int value, int index, int *assignment)
```

Deletion

Yêu cầu: Viết chương trình C đếm số bước **GÁN** trong thuật toán **Deletion** được cho ở mục 2

Lập **bảng tần số** và tính **trung bình cộng** số bước **GÁN** của thuật toán khi thực hiện trên mảng có kích cỡ cố định $n = 10$ được sinh ngẫu nhiên với giá trị của mỗi phần tử nằm trong khoảng $[-1000; 1000]$, value cần chèn là **k** và chỉ số cần chèn cũng được sinh ngẫu nhiên từ 0 đến $n-1$. Thực hiện yêu cầu với:

- $k = 100$
- $k = 1000$
- $k = 10000$

Chỉnh sửa hàm **delete** trên với khai báo:

```
void delete(int arr[], int* n, int index, int *assignment)
```

04 BÀI TẬP VỀ NHÀ

Big O trong một số thuật
toán nâng cao.

Bài 1: (*minIndex*)

Cho một mảng số thực **arr** có kích thước **n** và các giá trị **arr[i]** nằm trong đoạn $[-2*n; 2*n]$.

1. Hãy viết hàm tìm giá trị nhỏ nhất **int minIndex(float arr[], int n)** trong tất cả phần tử của mảng **arr[i]**. Hàm này trả về chỉ số nhỏ nhất của phần tử mang giá trị nhỏ nhất trong mảng.

Ví dụ: 8 7 3 1 2 3 2 1 9 1 2 1 thì hàm **minIndex** trả về giá trị là 3.

2. Cài đặt chương trình C để đếm số lần **so sánh** xuất hiện trong hàm **minIndex**. Sinh các mảng **arr** có kích thước **n = 10** ngẫu nhiên để tính số lần so sánh trung bình và lập bảng tần số thể hiện số lần so sánh sau 100, 1000 lần chạy thuật toán với các mảng sinh ngẫu nhiên.

Bài 2: (Student's scores)

Hiệu trưởng cung cấp cho nhân viên phân tích dữ liệu Nhung danh sách điểm thi giữa kỳ 1 môn Toán của học sinh, **chỉ gồm 1 dãy n số thực nằm trong đoạn [0;10]** và yêu cầu Nhung vẽ biểu đồ Box and Whisker để xem điểm số của học sinh phân tán như thế nào trong kỳ thi. Tuy nhiên, để vẽ được biểu đồ, Nhung phải sắp xếp dãy điểm của học sinh theo thứ tự tăng dần. Nhung mãng hoá các số liệu điểm rồi làm như sau:

Cho mảng **arr[]** có kích thước **n**.

- **Bước 1:** Duyệt mảng từ **i = 0** đến **i = n - 1**
- **Bước 2:** Trong các phần tử từ **arr[i]** đến **arr[n-1]**, tìm **minIndex**.
- **Bước 3:** Hoán đổi (swap) giá trị của **arr[minIndex]** và **arr[i]** rồi quay về **bước 1**.

Yêu cầu:

1. Viết hàm **sort** để sắp xếp mảng **arr** theo các bước làm trên. Cài đặt chương trình C để kiểm tra kết quả với mẫu **n = 40**.
2. Cải tiến chương trình trên bằng cách thêm vào biến đếm số lần so sánh **comparison** vào hàm **sort**. Sử dụng thống kê để ước lượng số lần **comparison** trung bình ứng với **n = 40**.
3. (*) Ước lượng **O(n)** của thuật toán sắp xếp mảng trên bằng thống kê.

Bài 3: (Bubble sort)

Hàm **bubbleSort** có chức năng sắp xếp một mảng bất kỳ theo thứ tự tăng dần.

1. Cài đặt chương trình C cho một mảng số nguyên có kích thước 20 với giá trị của các phần tử là ngẫu nhiên, hãy sử dụng hàm bubbleSort bên để sắp xếp mảng theo thứ tự tăng dần.
2. Hãy đếm số lần so sánh được dùng trong hàm **bubbleSort** với mảng trên, số lần sinh mảng ngẫu nhiên để khảo sát lần lượt là 10, 20, 50, 100, 1000, 10000. **Em có nhận xét gì về số lần comparison trung bình?**
3. Thực hiện giống với yêu cầu **b** với kích cỡ mảng thay đổi lần lượt là **n = 10, 20, 30, 40, 50, 60, 70, 80, 90, 100**. Em có nhận xét gì về mối liên hệ giữa kích thước mảng **n** và số bước trung bình **comparison** ứng với từng trường hợp của **n**? Từ đó ước lượng $O(n)$.
4. So sánh $O(n)$ của thuật toán bubble sort với thuật toán sắp xếp ở bài 3. Từ đó em rút ra được kết luận gì?

```
void bubbleSort(int arr[], int n, int *swaps){
    *swaps = 0;
    for(int i = 0; i < n; i++){
        for(int j = n-1; j > i; j--){
            if(arr[j-1] > arr[j])
                swap(arr+j-1, arr+j);
        }
    }
}
```

Bài 4: (Expired Year)

Có một mẫu dữ liệu gồm n số nguyên là các năm sản xuất của dòng xe BMW nthunglaom1620. Các nhà nghiên cứu thị trường quan tâm đến các mẫu xe sản xuất trước năm k nên họ yêu cầu hãng xe xóa tất cả dữ liệu năm từ năm k trở đi. Hãng xe được tư vấn 2 thuật toán như sau:

1. (caseA) Trực tiếp xét từng năm, nếu năm đó lớn hơn hoặc bằng k thì **delete** năm đó khỏi mẫu dữ liệu.
2. (caseB) Sắp xếp tất cả các năm thành một chuỗi tăng, **delete** tất cả các năm lớn hơn bằng k trở về sau.

Yêu cầu:

- a. Viết hai hàm, **caseA** và **caseB** để thực hiện hai cách làm trên. Cài đặt chương trình C, áp dụng cho mảng kích cỡ $n = 100$ với giá trị phần tử lấy ngẫu nhiên từ tập $[1998 ; 2023]$ và $k = 2016$.
- b. Vận dụng những phương pháp đánh giá đã học để đánh giá thuật toán nào tốt hơn theo tiêu chuẩn đếm nào? Giải thích vì sao.



Bài 5: (negative sum)

Viết hàm tính tổng các giá trị âm trong mảng a có n số nguyên.



Thêm biến vào tham số của hàm trên, để đếm số phép gán khi thực hiện phép tính cộng dồn các giá trị âm trong mảng.

Viết hàm main() để thực hiện 1000 lần hàm trên với mảng a có 10 phần tử có giá trị ngẫu nhiên trong $[-100, 100]$. Hãy tính tần suất xuất hiện của số phép gán của hàm trên.



Lưu ý:

- Không cần kiểm tra n là số nguyên dương.
- Ghi kết quả xuất ra của chương trình vào cuối bài làm.

Bài 6: (*maximum product*)

Cho một mảng số nguyên dương có N phần tử. Viết hàm tìm tích lớn nhất giữa 2 phần tử trong mảng.

Ví dụ: mảng [4, 1, 3, 2] có tích $4 \times 3 (=12)$ là lớn nhất trong các tích giữa 2 phần tử trong mảng.

Gợi ý thuật toán: Giả định tích lớn nhất max là tích giữa 2 phần tử đầu tiên. Sau đó, lần lượt lấy một phần tử trong mảng, rồi tính tích product với một phần tử còn lại, và so sánh với max. Nếu lớn hơn thì gán max là product.

1. Hãy chèn code để tính số lần (steps) tính tích giữa 2 phần tử trong mảng. Hãy cho biết số steps chính xác theo tham số N (số phần tử của mảng) và theo Big O.
2. Hãy chèn code để đếm số lần (steps: assignments) gán $\text{max} = \text{product}$ ở trên (số lần cập nhật max khi tìm được tích lớn nhất). Sau đó, viết chương trình chính chạy thực nghiệm 10.000 lần, với mỗi lần tạo một mảng nguyên dương có 4 phần tử có giá trị ngẫu nhiên trong đoạn [1,100]. Hãy thống kê số assignment trung bình là bao nhiêu, và tần suất xuất hiện của nó.

Thực hiện yêu cầu 1 và yêu cầu 2 trên cùng một source code.

Bài 7: (*ascending integers*)

Viết hàm kiểm tra một số nguyên dương n có các chữ số tăng dần từ trái sang phải hay không. Ví dụ: 1234, 3345 là các số nguyên dương có các chữ số tăng dần từ trái sang phải.

Thêm biến vào tham số của hàm trên, để đếm số phép so sánh khi so sánh 2 chữ số kế nhau của số nguyên dương n .

Ví dụ:

- Kiểm tra số 2354, khi so sánh 5 với 4 thì chúng ta đếm là một phép so sánh. Và do $5 > 4$ (không tăng dần) nên hàm kết thúc với 1 phép so sánh.
- Kiểm tra số 2323, chúng ta so sánh 2 với 3, rồi 3 với 2 thì hàm kết thúc với 2 phép so sánh vì $3 > 2$.

Viết hàm main() để tính trung bình của **số phép so sánh** của hàm trên với tham số n là tất cả các số từ 1000 đến 9999.

Lưu ý:

- Không cần kiểm tra n là số nguyên dương.
- Ghi kết quả xuất ra của chương trình vào cuối bài làm.

Bài 8: (shell sort)

Đếm số phép gán (assigns) và dịch chuyển (shifts) theo từng vòng và tổng kết của thuật toán sau (chỉ đếm số phép gán của các biến liên quan đến phần tử của mảng a)

```
void f(int a[], int n) {
    for (int d = n/2; d > 0; d /= 2) {
        for (int i = d; i < n; i++) {
            int t = a[i];
            int j;
            for (j = i; j >= d && a[j-d] > t; j -= d) {
                a[j] = a[j-d];
            }
            a[j] = t;
        }
    }
}

int main() {
    int a[] = {34, 29, 10, 3, 2, 44, 88, 1, 21, 12, 3, 18, 21, 23, 37, 17, 8, 9, 4, 0};
    int n = 20;
    f(a,n);
    return 0;
}
```



The end!



Does anyone have any questions?

ta.tinthh.hcmus@gmail.com